

VOXGEST: A BIDIRECTIONAL DIGITAL ACCESSIBILITY APPLICATION FOR SIGN
LANGUAGE RECOGNITION AND REAL-TIME SPEECH COMMUNICATION USING
DEEP LEARNING AND VISUAL TRACKING

A Capstone Project Presented to
College of Computer Studies
Our Lady of Fatima University - Quezon City

In Partial Fulfillment for the Requirements
for the Degree of
Bachelor of Science in Information Technology

Martinez, Jhon Cedrick D.
Almodiel, Anastacia Reiza G.
Manaog, Earle Cassey L.
Teofilo, Mariella S.
July 2026

APPROVAL SHEET

This capstone project VOXGEST: A BIDIRECTIONAL DIGITAL ACCESSIBILITY APPLICATION FOR SIGN LANGUAGE RECOGNITION AND REAL-TIME SPEECH COMMUNICATION USING DEEP LEARNING AND VISUAL TRACKING presented orally and submitted by MARTINEZ, JHON CEDRICK D.; ALMODIEL, ANASTACIA REIZA G.; MANAOG, EARLE CASSEY L.; and TEOFILO, MARIELLA S. has been approved and accepted as partial fulfillment of the requirements for the degree of BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY.

Mr. Joseph Martin Gojo, MIT, LPT
Research Adviser

COMMITTEE ON ORAL EXAMINATION

Name of Chair, Title
Chair/Critic

Ms. Emmie Faye Matabile, MIT
Panel Member

Mr. Johani Basaula, MSIT
Panel Member

Prof. Raymond S. Macatangga, DIT
Dean

ENDORSEMENT

This capstone project entitled: VOXGEST: A BIDIRECTIONAL DIGITAL ACCESSIBILITY APPLICATION FOR SIGN LANGUAGE RECOGNITION AND REAL-TIME SPEECH COMMUNICATION USING DEEP LEARNING AND VISUAL TRACKING prepared by Jhon Cedrick D. Martinez, Anastacia Reiza G. Almodiel, Earle Cassey L. Manaog, and Mariella S. Teofilo of BSIT 3-Y2-6, in partial fulfillment of the requirements for the degree Bachelor of Science in Information Technology has been examined and is now recommended for Oral Examination.

This is to certify that Jhon Cedrick D. Martinez, Anastacia Reiza G. Almodiel, Earle Cassey L. Manaog, and Mariella S. Teofilo are ready for the Oral Examination.

Mr. Joseph Martin Gojo, MIT, LPT
Research Adviser

TABLE OF CONTENTS

	Page
Approval Sheet	ii
Endorsement	iii
Table of Contents.....	iv
List of Tables.....	vi
List of Figures	vii
1 CHAPTER 1 - INTRODUCTION	1
Background of the Study.....	1
Project Context	2
Purpose and Description	4
Statement of the Problem.....	5
Objectives of the Study	6
Scope and Limitation	7
2 CHAPTER 2 - REVIEW OF RELATED LITERATURE AND STUDIES	9
Sign Language Recognition and Assistive Communication	9
Deep Learning for Static Gesture Recognition	10
Landmark-Based Hand Tracking and MediaPipe	10
Dynamic Sign Recognition Using Sequential Models	11
Negative Class and Recognition Hardening.....	12
Edge Computing and Android-Based Deployment.....	13
Bidirectional Communication, Speech-to-Text, and Avatar Feedback	14
Software Quality Evaluation Using ISO/IEC 25010	14
Synthesis and Relevance to the Present Study	15

3 CHAPTER 3 - METHODOLOGY	16
Research Design	16
Development Methodology	16
System Architecture	17
System Components	18
Dataset Development	19
Static Alphabet Dataset	19
Dynamic Word Dataset.....	20
Negative and No-Output Samples.....	20
Feature Engineering	20
Static Feature Engineering.....	21
Dynamic Feature Engineering	21
Model Development	22
Initial CNN Approach	22
Hybrid CNN and Landmark Attempt	22
Landmark-Based Static Model	22
Dynamic LSTM and TCN Models	22
Recognition Hardening and Post-Processing	23
Android Application Development	24
Avatar and Speech Communication Module	25
Testing Procedure	25
Static Recognition Testing	25
Dynamic Word Testing	25
Negative Class Testing	26
Android Readiness Testing	26
Evaluation Method.....	26
Respondents of the Study	27
Data Gathering Instrument	27
Statistical Treatment of Data.....	27
Research Ethics	28
Summary of Methodology	28
BIBLIOGRAPHY	30

LIST OF TABLES

TABLE	Page
3.1 Likert Scale Interpretation.....	27

LIST OF FIGURES

FIGURE	Page
3.1 VoxGest System Design Architecture	18

CHAPTER 1 - INTRODUCTION

This chapter presents the problem and background of the study. It discusses the communication barriers experienced by Deaf, mute, speech-impaired, and hard-of-hearing individuals, the need for a portable assistive communication tool, and the proposed development of VoxGest as an Android-based accessibility application using visual tracking, deep learning, and on-device inference.

Background of the Study

Communication is a fundamental part of human interaction, education, healthcare, public service, employment, and daily social participation. However, individuals who rely on sign language often encounter communication barriers when interacting with hearing or non-signing people. These barriers become more serious in situations where a professional interpreter is unavailable, such as hospital visits, pharmacies, public offices, schools, interviews, emergency counters, and ordinary face-to-face conversations. Although interpreters and assistive tools exist, they are not always accessible, affordable, immediate, or practical in every daily interaction.

The development of artificial intelligence, computer vision, mobile computing, and edge-based machine learning provides an opportunity to create assistive systems that can recognize hand gestures through a camera and transform recognized gestures into readable or audible outputs. Sign language recognition systems are especially relevant because they can support communication between signing and non-signing individuals without requiring specialized external sensors, gloves, or continuous internet connectivity.

VoxGest is proposed as a bidirectional digital accessibility application for sign language recognition and real-time speech communication. The system is designed to use a mobile device camera to detect hand and body landmarks, process the extracted features through trained recognition models, and convert confirmed sign inputs into text and

speech. To support the reverse communication flow, the application also includes speech-to-text input and a lightweight avatar or fingerspelling response for the signing user. This makes the project more than a one-way sign-to-speech converter; it is intended to support a basic conversation flow between a sign language user and a hearing or non-signing person.

The current development of VoxGest focuses on a controlled and realistic recognition scope. Static alphabet and control signs are recognized through a hand-landmark model, while selected dynamic word signs are recognized through temporal landmark sequences. The active dynamic vocabulary is limited to commonly useful words such as YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, and THANKYOU, with NOTHING used as a negative or no-output class. This controlled scope allows the researchers to prioritize reliability, reduce false outputs, and prepare the system for Android deployment before expanding to larger vocabulary or full sentence-level interpretation.

A major accessibility decision in VoxGest is its dominant-hand recognition policy. Instead of requiring full two-hand signing at the current stage, the system prioritizes one configured hand for recognition. This design supports users who sign with one dominant hand by preference or by necessity, including users with limb difference, injury, amputation, stroke-related mobility limitations, or other physical restrictions. Therefore, VoxGest is not only a sign recognition project but also an accessibility-oriented communication tool designed around practical use, portability, and inclusive interaction.

Project Context

VoxGest is developed within the fields of artificial intelligence, deep learning, computer vision, Android mobile development, and digital accessibility. The system is intended to operate as an offline Android prototype that performs recognition locally on the device using TensorFlow Lite models. This design is important because a communication tool for public and daily environments should not depend fully on cloud services or stable internet connectivity. Local processing also supports privacy because camera-based gesture

recognition can be performed on the device rather than sending user video to an external server.

The current project direction differs from the earlier concept of a purely CNN-based static alphabet recognizer. The project initially explored image-based CNN recognition because CNNs are commonly used for visual classification. However, live testing showed that raw image recognition was sensitive to background, lighting, hand size, camera distance, and cropping conditions. Because of these limitations, the system evolved toward landmark-based feature extraction using MediaPipe. This shift allowed the model to focus on hand and body geometry rather than the full camera image.

The present VoxGest pipeline is composed of multiple recognition and communication modules. The static alphabet module recognizes A-Z, del, space, and nothing through 63 hand-landmark features. The dynamic word module recognizes selected motion signs using a 30-frame sequence of 162 features per frame, composed of pose and dominant-hand landmarks. The system also applies post-processing gates such as confidence, margin, motion, wrist path, and hand presence before accepting predictions. Accepted outputs are passed to a token composer that builds the displayed message. NOTHING is treated as a no-output class and is not spoken, displayed, or animated as a word.

The Android implementation is designed around the same runtime behavior used in the Python prototype. The recommended mobile stack includes Kotlin, CameraX, MediaPipe Holistic or an equivalent landmark pipeline, and TensorFlow Lite interpreters for the static and dynamic recognition models. The Android application is expected to include camera input, recognition modes, text output, speech output, speech-to-text, and avatar playback. Phrase-level recognition has been explored as a future pathway, but the current defense-ready priority is recognition hardening: static alphabet, 10 dynamic words, NOTHING, token-based sentence output, and avatar playback from confirmed tokens.

Purpose and Description

The overall purpose of this study is to design and develop VoxGest, a bidirectional digital accessibility application that assists communication between sign language users and hearing or non-signing individuals. The system aims to recognize selected sign inputs from a camera, convert accepted signs into text and speech, and provide a visual response path through speech-to-text and avatar or fingerspelling output.

Specifically, the VoxGest application is designed to provide the following capabilities:

1. Capture live visual input using an Android mobile device camera for real-time gesture tracking;
2. Extract hand and upper-body landmarks using MediaPipe-based visual tracking;
3. Recognize static alphabet and control signs using normalized hand-landmark features;
4. Recognize selected dynamic word signs using temporal pose and dominant-hand landmark sequences;
5. Reduce false recognition by using a NOTHING class and acceptance gates before output is displayed or spoken;
6. Compose readable message output from confirmed accepted tokens such as letters, control signs, and word signs;
7. Convert recognized text into speech output for the hearing or non-signing listener;
8. Convert spoken replies into text through speech-to-text; and
9. Provide a lightweight avatar or fingerspelling response for known and unknown words.

The application is designed as an assistive communication prototype and not as a complete replacement for professional interpreters. Its purpose is to demonstrate how computer vision, deep learning, and on-device Android deployment can be used to create a practical communication bridge in controlled real-world situations where immediate interpretation support is not available.

Statement of the Problem

The general problem of this study is the communication barrier between sign language users and hearing or non-signing individuals in situations where an interpreter or reliable assistive communication support is not immediately available. To address this problem, the study aims to develop VoxGest, a bidirectional Android-based accessibility application for selected sign language recognition, text and speech output, speech-to-text response, and avatar-based visual feedback.

Specifically, this study seeks to answer the following questions:

1. How can an Android-based accessibility application be designed and developed to capture live camera input and perform on-device sign recognition?
2. How can MediaPipe landmark tracking be used to extract hand and body features for static alphabet and dynamic word recognition?
3. How can the system implement static ASL alphabet recognition using normalized 63-feature hand-landmark inputs?
4. How can the system implement selected dynamic word recognition using 30-frame temporal sequences with 162 pose and dominant-hand features per frame?
5. How can the system reduce false outputs through acceptance gates, a NOTHING negative class, and token-based sentence composition?
6. How can VoxGest support bidirectional communication through sign-to-text, text-to-speech, speech-to-text, and avatar or fingerspelling response?

7. What is the level of software quality of the developed VoxGest system based on ISO/IEC 25010, as evaluated by IT professionals and target users, in terms of:

7.1 Functional Suitability;

7.2 Performance Efficiency;

7.3 Reliability; and

7.4 Usability?

Objectives of the Study

The main objective of this study is to design and develop VoxGest, a bidirectional digital accessibility application for selected sign language recognition and real-time speech communication using deep learning and visual tracking.

Specifically, the study aims to:

1. Design and develop an Android-based accessibility application that uses camera input for real-time sign recognition;
2. Implement MediaPipe-based landmark extraction for hand and upper-body tracking;
3. Implement static alphabet and control sign recognition for A-Z, del, space, and nothing using 63 normalized hand-landmark features;
4. Implement selected dynamic word recognition for YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, THANKYOU, and NOTHING using temporal landmark sequences;
5. Apply post-processing gates, token composition, and a NOTHING negative class to reduce false or unstable recognition outputs;
6. Integrate text output and text-to-speech for sign-to-speech communication;
7. Integrate speech-to-text and avatar or fingerspelling playback for hearing-user replies;

8. Prepare the recognition models for offline Android deployment using TensorFlow Lite; and
9. Evaluate the software quality of VoxGest using ISO/IEC 25010 in terms of Functional Suitability, Performance Efficiency, Reliability, and Usability.

Scope and Limitation

The scope of this study covers the design, development, and evaluation preparation of VoxGest as an Android-based assistive communication prototype. The system focuses on camera-based recognition using hand and body landmarks, static alphabet recognition, selected dynamic word recognition, token-based sentence composition, text-to-speech, speech-to-text, and lightweight avatar or fingerspelling output. The recognition scope is limited to A-Z, del, space, nothing, and the current dynamic words YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, THANKYOU, and NOTHING.

The system is designed to run locally on Android using TensorFlow Lite models and does not require a backend server for the recognition process. Model training, dataset preparation, and model export are performed in the Python development environment, while the Android application consumes exported model files, label files, and runtime configuration. The Android build is expected to mirror the Python feature extraction and post-processing behavior to maintain consistency between training and deployment.

The study does not claim to translate unrestricted ASL grammar, full conversations, or all possible sign language vocabulary. Phrase-level sentence recognition has been explored as a future pathway, but it is disabled by default during the current recognition-hardening stage. The avatar module is also limited to lightweight visual feedback, known word animations, placeholder animations, and fingerspelling fallback. It does not claim to provide complete or linguistically perfect ASL avatar translation.

The performance of the system may still be affected by camera quality, lighting conditions, distance from the camera, hand visibility, signer speed, body position, and con-

sistency between training samples and live usage. The dynamic word recognition model is considered demo-ready and still requires additional live testing and repair samples for weak or confusing signs. Therefore, VoxGest is intended as an assistive communication prototype for controlled use and evaluation, not as a replacement for certified human interpreters in legal, medical, educational, or emergency-critical communication.

CHAPTER 2 - REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents literature and studies related to sign language recognition, computer vision, landmark tracking, dynamic gesture modeling, edge deployment, bidirectional assistive communication, avatar-based feedback, and software quality evaluation. The reviewed works provide the technical and conceptual foundation for VoxGest and explain why the system uses a landmark-based, on-device, and accessibility-oriented approach.

Sign Language Recognition and Assistive Communication

Sign language recognition is a computer vision and machine learning task that aims to interpret visual language inputs and convert them into a form that can be understood by non-signers. It is often developed as an assistive technology because it can reduce communication barriers between Deaf or hard-of-hearing individuals and hearing people. Rastgoo et al. (2021) described sign language recognition as a broad area that involves hand shape, motion, body pose, and sometimes facial expression, making it more complex than ordinary image classification. Their survey emphasized that both spatial and temporal features are important because signs are not always defined by a single hand image.

Bragg et al. (2019) also explained that sign language technology should be viewed from an interdisciplinary perspective because recognition, generation, and translation systems must consider language structure, user interaction, accessibility, and the lived experience of sign language users. This is important for VoxGest because the project is not only a machine learning classifier but also an accessibility tool. The system must produce outputs that are useful in communication, not only predictions that are correct in a dataset.

The current VoxGest system follows this assistive direction by focusing on selected signs that can support practical communication. Instead of claiming unrestricted sign lan-

guage translation, the system recognizes a controlled vocabulary and uses text, speech, and avatar or fingerspelling feedback to support basic interaction between signing and non-signing users.

Deep Learning for Static Gesture Recognition

Deep learning has been widely used for gesture recognition because neural networks can learn visual and geometric patterns from gesture data. Convolutional Neural Networks (CNNs) are commonly used for image-based recognition because they can identify edges, shapes, and visual patterns from raw images. Kasapbasi et al. (2022) developed DeepASLR, a CNN-based interface for American Sign Language recognition, showing that CNN models can classify static signs effectively when trained with controlled image data.

Alzubaidi et al. (2023) also demonstrated the use of deep learning and transfer learning for American Sign Language recognition. Transfer learning can reduce training difficulty by using pre-trained models that already learned useful visual features. These studies support the original idea of VoxGest, where the project first explored image-based CNN recognition for static alphabet signs.

However, static image recognition alone has limitations in live camera environments. The current development of VoxGest found that raw image-based CNN recognition was affected by lighting, background, cropping, camera angle, and differences between dataset images and webcam conditions. Therefore, CNN-based recognition is discussed in this study as an important starting point and related approach, but not as the only final recognition method of the current system. VoxGest moved toward landmark-based static recognition because landmarks provide a more stable representation of hand geometry.

Landmark-Based Hand Tracking and MediaPipe

Landmark-based tracking is important for VoxGest because it converts camera frames into numerical coordinates that represent the hand and body. Instead of asking the model

to understand the entire camera image, the system focuses on the points that matter for signing. Zhang et al. (2020) introduced MediaPipe Hands as a real-time, on-device hand tracking pipeline that predicts hand skeleton landmarks from a single RGB camera. Their work supports the feasibility of extracting hand landmarks in real-time environments without requiring external sensors.

Lugaresi et al. (2019) presented MediaPipe as a framework for building perception pipelines. MediaPipe is relevant to VoxGest because the system needs a reusable and real-time visual tracking layer that can operate across development and mobile environments. For static alphabet recognition, VoxGest uses 21 hand landmarks with x, y, and z coordinates. This produces 63 features per hand frame. The features are normalized around the wrist and scaled by hand size, helping the model recognize signs despite changes in camera distance or hand position.

For dynamic word recognition, VoxGest uses MediaPipe Holistic or an equivalent pose-and-hand landmark pipeline. This is necessary because some signs depend on body location. A hand movement near the face can mean something different from a similar movement near the chest. By including pose landmarks and dominant-hand landmarks, the model gains body-relative spatial context that cannot be captured by hand landmarks alone.

Dynamic Sign Recognition Using Sequential Models

Dynamic signs require temporal modeling because their meaning depends on movement across time. A single image cannot fully represent signs such as HELLO, THANKYOU, WATER, STOP, or PLEASE because these signs involve motion, direction, repetition, or location changes. Hochreiter and Schmidhuber (1997) introduced Long Short-Term Memory networks to address sequence learning problems, making LSTM suitable for recognizing gesture sequences where earlier frames influence later interpretation.

Li et al. (2020) introduced the Word-Level American Sign Language dataset, commonly known as WLASL, which contains large-scale video data for word-level ASL recog-

nition. Their work showed that word-level recognition is more challenging than static image classification because the model must learn both appearance and motion patterns. WLASL is relevant to VoxGest because the project uses dynamic word videos and webcam calibration samples to train selected word signs.

Temporal Convolutional Networks are also relevant for sequence modeling. Bai, Kolter, and Koltun (2018) found that temporal convolutional architectures can be strong alternatives to recurrent models for sequence tasks. VoxGest includes LSTM and TCN directions because both architectures can learn motion patterns from temporal landmark sequences. The current system compares these models based not only on validation accuracy but also on live behavior, since live camera recognition may reveal failures not visible in saved-data testing.

The present VoxGest dynamic word model uses a 30-frame sequence. Each frame contains 162 features composed of 99 pose features and 63 dominant-hand features. This design allows the system to process both the motion of the hand and the location of the sign relative to the body.

Negative Class and Recognition Hardening

Real-time recognition systems may produce false outputs if every camera movement is forced into one of the known classes. In a softmax classifier, the model will always select a top label even when the user is idle, transitioning between signs, moving casually, or not signing at all. For this reason, VoxGest includes NOTHING as a negative or no-output class.

The purpose of NOTHING is to teach the system what non-sign input looks like. Examples include idle hands, open hands, aborted signs, partial signs, transition movements, and hand entering or leaving the frame. This supports reliability because the system should not speak, display, or animate a word unless the prediction is stable and meaningful. In VoxGest, the NOTHING class works together with post-processing gates that check confidence, prediction margin, motion, wrist path, and hand presence before a sign

is accepted.

Recognition hardening is therefore part of the methodology, not just a debugging task. The system must be tested live, logged, repaired with additional data, and retrained when weak signs are discovered. This approach supports the defense of VoxGest because it shows that the researchers understand the difference between dataset accuracy and practical live reliability.

Edge Computing and Android-Based Deployment

Edge computing allows intelligent processing to happen directly on a user device instead of depending on a remote server. This is important for assistive communication because the user may need the application in places where internet access is weak, unavailable, or inappropriate for privacy reasons. Warden and Situnayake (2020) explained that TensorFlow Lite and TinyML approaches allow machine learning models to run on constrained devices, supporting low-latency and local inference.

Abadi et al. (2016) introduced TensorFlow as a flexible machine learning system, while TensorFlow Lite later became a common deployment pathway for mobile and embedded devices. For VoxGest, model training is done in the Python workspace, while Android consumes exported TensorFlow Lite models, labels, and runtime configuration. This separation allows the researchers to train and test models on a development machine while preparing the Android app for on-device recognition.

The Android direction of VoxGest uses Kotlin, CameraX, MediaPipe landmark extraction, TensorFlow Lite interpreters, and local text-to-speech and speech-to-text components. This supports the goal of an offline communication prototype. The Android application is not expected to retrain models; it only performs live inference using exported model files.

Bidirectional Communication, Speech-to-Text, and Avatar Feedback

A one-way sign-to-speech tool helps the hearing person understand the sign language user, but it does not fully solve conversation. The hearing person must also be able to respond in a way that the sign language user can access. For this reason, VoxGest includes a bidirectional communication flow.

The first direction is sign-to-text and sign-to-speech. The signing user performs a recognized sign, the app displays the confirmed output as text, and the text can be spoken aloud through text-to-speech. The second direction is speech-to-text and avatar response. The hearing person speaks into the device, the app converts speech into readable text, and known words can be shown through avatar animation or unknown words can be fingerspelled.

The avatar module in VoxGest is intentionally lightweight. It does not claim complete ASL grammar generation or full linguistic signing correctness. Instead, it provides visual feedback for known words and fingerspelling fallback for unknown words. This supports accessibility while keeping the scope realistic for the current capstone stage.

Software Quality Evaluation Using ISO/IEC 25010

ISO/IEC 25010 provides a software quality model that can be used to evaluate whether a system meets its required qualities. For VoxGest, the selected quality characteristics are Functional Suitability, Performance Efficiency, Reliability, and Usability. These criteria fit the system because the application must perform the required communication tasks, respond fast enough for live use, avoid unstable or false outputs, and remain understandable to target users.

Functional Suitability evaluates whether VoxGest performs its intended functions, including camera-based recognition, text output, speech output, speech-to-text, token composition, and avatar response. Performance Efficiency evaluates response time, inference behavior, and mobile suitability. Reliability evaluates stability under real use, including

hand tracking failure, missing landmarks, false triggers, and no-output handling. Usability evaluates whether Deaf, mute, hard-of-hearing, and hearing users can understand and operate the application with minimal difficulty.

The study will use a 5-point Likert scale questionnaire to collect evaluation data from IT professionals and target or representative users. This evaluation supports the research objective of measuring both technical quality and communication usefulness.

Synthesis and Relevance to the Present Study

The reviewed literature supports the current design of VoxGest in several ways. Studies on deep learning and CNNs justify the original exploration of image-based static gesture recognition. Studies on MediaPipe and landmark tracking support the decision to use hand and body landmarks instead of relying entirely on raw images. Studies on LSTM, TCN, and WLASL justify the use of temporal sequence models for dynamic word recognition. Edge computing literature supports the use of TensorFlow Lite and on-device Android inference. Literature on accessibility and sign language technology supports the need for bidirectional communication, while ISO/IEC 25010 supports the evaluation method.

The present study adapts these ideas into a controlled and realistic capstone system. VoxGest does not claim unrestricted ASL translation. Instead, it demonstrates a defense-ready accessibility prototype with static alphabet recognition, selected dynamic word recognition, negative-class rejection, token-based sentence composition, text-to-speech, speech-to-text, and lightweight avatar or fingerspelling feedback. This scope is appropriate because the system prioritizes reliability, offline use, and practical communication before expanding to larger vocabulary or full sentence-level sign language interpretation.

CHAPTER 3 - METHODOLOGY

This chapter presents the research design, development methodology, system architecture, feature extraction process, model development process, Android deployment plan, testing procedure, and evaluation method used in the development of VoxGest. The methodology is written to explain what was developed, why each approach was selected, and how each component is implemented in relation to the objectives of the study.

Research Design

This study uses a developmental research design. Developmental research is appropriate because the study focuses on the creation, improvement, and evaluation of a working technology artifact. The output of the study is not limited to theoretical analysis; it includes the design and development of VoxGest as an Android-based assistive communication prototype.

The system was developed to address a practical communication problem between sign language users and hearing or non-signing individuals. The research process therefore includes requirement analysis, system design, machine learning model development, live recognition testing, Android handoff preparation, and software quality evaluation. The system will be evaluated using ISO/IEC 25010 quality characteristics, specifically Functional Suitability, Performance Efficiency, Reliability, and Usability.

Development Methodology

The development of VoxGest follows Agile Rapid Application Development. This methodology was selected because the project requires continuous testing, revision, and refinement. Sign language recognition systems are sensitive to camera quality, lighting, background, hand visibility, motion timing, body position, and differences between saved training data and live usage. A strictly linear development process would not be suitable because the system must be tested repeatedly in real camera conditions.

The Agile RAD process used in VoxGest consists of the following phases:

1. Brainstorming and problem identification;
2. Requirement analysis and scope definition;
3. Dataset preparation and sample collection;
4. Feature extraction and preprocessing;
5. Model development and export;
6. Live recognition testing and hardening;
7. Android prototype integration;
8. Evaluation preparation; and
9. Documentation and research paper refinement.

This iterative approach allowed the researchers to improve the system when earlier models showed weaknesses. The project initially explored image-based CNN recognition, then attempted a hybrid image-and-landmark approach, and later shifted toward landmark-based static recognition and temporal dynamic recognition because these approaches matched the live camera requirements more effectively.

System Architecture

VoxGest is designed as an offline Android edge-computing accessibility system. The general system flow is:

```
camera input -> landmarks -> static/dynamic model -> gates ->  
token composer -> text/speech/avatar
```

The system begins with camera input from the Android device. Frames are processed through a landmark extraction pipeline. Static signs are passed to the static alphabet

recognition model, while dynamic words are passed to the temporal word recognition model. Predictions are not immediately accepted; they are filtered through confidence, margin, motion, wrist path, hand presence, stability, and cooldown rules. Accepted predictions are passed to the token composer, which builds the final text output. The text can then be displayed, spoken through text-to-speech, or passed to the avatar module.

For the reverse communication direction, the hearing user can speak into the application. The speech-to-text module converts the spoken reply into text. Known words may trigger avatar animations, while unknown words can be fingerspelled. This creates a bidirectional communication flow between the signing user and the hearing or non-signing user.

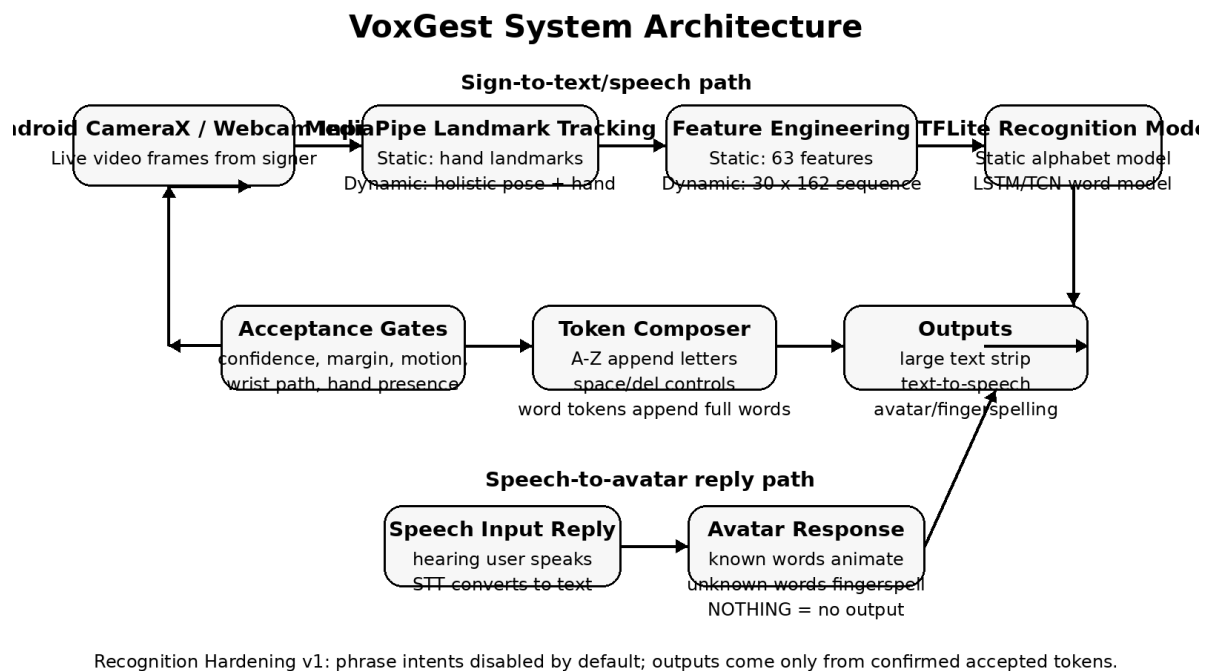


Figure 3.1: VoxGest System Design Architecture

System Components

The VoxGest system is composed of the following components:

1. **Camera Input Module** - captures live video frames from the Android camera or

webcam prototype.

2. **Landmark Extraction Module** - extracts hand and pose landmarks using MediaPipe-based tracking.
3. **Static Alphabet Recognition Module** - recognizes A-Z, del, space, and nothing using 63 hand-landmark features.
4. **Dynamic Word Recognition Module** - recognizes selected motion words using 30-frame sequences with 162 features per frame.
5. **Post-Processing and Acceptance Module** - prevents unstable predictions using thresholds, smoothing, gates, and cooldown rules.
6. **Token Composer Module** - builds message text only from confirmed accepted letters, controls, and word tokens.
7. **Text-to-Speech Module** - speaks confirmed recognized output for the hearing or non-signing user.
8. **Speech-to-Text Module** - captures spoken replies and converts them into readable text.
9. **Avatar and Fingerspelling Module** - provides visual feedback using known word animations or fingerspelling fallback.

Dataset Development

The dataset development process was divided into static alphabet data, dynamic word data, negative/no-output data, and future phrase-intent data.

Static Alphabet Dataset

The static alphabet dataset was prepared for A-Z, del, space, and nothing. The first version of the project used image datasets for CNN-based recognition. However, live

testing showed that image-based recognition did not generalize well under webcam conditions. For this reason, the project moved toward a landmark-based dataset where each sample represents the geometry of the hand instead of the entire image.

For static recognition, each hand sample contains 21 landmarks with x, y, and z coordinates. This produces 63 numerical features. The model is trained to classify these features into static alphabet and control labels.

Dynamic Word Dataset

The dynamic dataset contains selected word signs that require movement over time. The active dynamic labels are YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, THANKYOU, and NOTHING. Each dynamic sample is represented as a 30-frame sequence. Each frame contains pose and dominant-hand features so the model can understand both hand motion and body-relative location.

Dynamic samples may come from selected videos, WLASL-related sources, and manual webcam calibration recordings. Live calibration is important because a model trained only on external videos may not match the actual camera, signer, angle, lighting, and motion style used in the application.

Negative and No-Output Samples

The NOTHING class is collected as a negative class. It includes idle hands, relaxed hands, open hands, hand entering or leaving the frame, transition movements, partial signs, aborted signs, and neutral non-sign movement. NOTHING is not a word token. It is used to reduce false outputs by teaching the model when no valid sign should be displayed, spoken, or animated.

Feature Engineering

Feature engineering converts camera-based visual input into numerical data that can be processed by the recognition models.

Static Feature Engineering

For static alphabet recognition, the system uses 21 hand landmarks. Each landmark has x, y, and z coordinates:

$$21 \text{ landmarks} \times 3 \text{ coordinates} = 63 \text{ features}$$

The static feature vector is normalized by subtracting the wrist landmark from every point and scaling the hand using a stable hand-size reference. This makes the same sign more consistent even when the hand is closer, farther, or slightly shifted in the camera view.

Dynamic Feature Engineering

For dynamic word recognition, the system uses a 30-frame temporal sequence. Each frame contains:

$$33 \text{ pose landmarks} \times 3 = 99 \text{ features}$$

$$21 \text{ dominant-hand landmarks} \times 3 = 63 \text{ features}$$

$$99 + 63 = 162 \text{ features per frame}$$

The dynamic model input is therefore:

$$[1, 30, 162]$$

The pose and hand features are anchored to the pose nose point to provide body-relative context. This design helps differentiate signs that may have similar hand shape or movement but occur in different body locations. The system also supports a strict single-hand feature policy that keeps the head, torso, selected arm, and selected hand while masking non-dominant arm and unrelated lower-body landmarks.

Model Development

Initial CNN Approach

The first model approach used a CNN trained on static ASL alphabet images. This was selected because CNNs are commonly used for image recognition and can learn visual patterns from raw images. However, live webcam testing revealed that image-based recognition was sensitive to background, lighting, cropping, camera distance, and dataset mismatch. The CNN approach is therefore documented as an early research iteration rather than the final system approach.

Hybrid CNN and Landmark Attempt

A hybrid model was later considered to combine raw image features with hand landmark features. This was intended to help distinguish visually similar signs. However, the approach introduced more preprocessing complexity and remained vulnerable to image-branch problems. Because the goal of the project is reliable real-time mobile recognition, the system shifted toward a simpler and more controllable landmark-based pipeline.

Landmark-Based Static Model

The current static recognition approach uses normalized 63-feature hand landmarks. This was chosen because hand landmarks reduce background and lighting dependence and make the Python and Android pipelines easier to align. The static model recognizes A-Z, del, space, and nothing.

Dynamic LSTM and TCN Models

The dynamic word model uses temporal sequence learning because dynamic signs are defined by motion over time. LSTM is used because it can learn sequence order and motion dependencies. TCN is also considered because temporal convolution can be efficient for sequence modeling and may perform well in mobile deployment. VoxGest compares model behavior using both validation results and live camera tests.

The dynamic model currently uses a 30-frame input with 162 features per frame and recognizes the current 10-word vocabulary plus NOTHING. The current dynamic recognition stage is considered recognition hardening, meaning that the project prioritizes improving the existing vocabulary before adding more signs.

Recognition Hardening and Post-Processing

Recognition hardening is the process of improving live reliability after model training. The system does not immediately trust a raw prediction. Instead, it checks whether a prediction is confident, stable, and supported by enough motion and hand presence.

The post-processing gates include:

1. **Confidence** - the probability score of the predicted class;
2. **Margin** - the separation between the top prediction and the next likely class;
3. **Motion Energy** - whether enough movement occurred for a dynamic word;
4. **Wrist Path** - whether the hand traveled enough distance during the sequence;
5. **Hand Presence** - whether the configured hand was visible enough during the sequence;
6. **Stable Frames** - whether the prediction remains consistent across multiple frames; and
7. **Cooldown** - a waiting period that prevents repeated duplicate outputs.

These gates are important because the model may otherwise produce random words during idle hands, partial signs, or camera jitter. The NOTHING class and the acceptance gates work together to prevent unstable predictions from becoming text, speech, or avatar output.

Android Application Development

The Android application is designed to consume exported model files and perform on-device inference. The recommended Android stack includes Kotlin, CameraX for camera preview and frame analysis, MediaPipe Holistic or an equivalent landmark pipeline, and TensorFlow Lite Interpreter for static and dynamic models.

The Android application should provide the following screens and features:

1. Camera permission and startup screen;
2. Live recognition screen;
3. Recognition mode switch for AUTO, LETTERS, and WORDS;
4. Large current prediction display;
5. Text strip for recognized output;
6. Clear or delete controls;
7. Optional debug metrics for development builds;
8. Speech output for confirmed tokens;
9. Speech-to-text input for hearing-user replies; and
10. Avatar or fingerspelling response panel.

The Android app must match the Python preprocessing rules. If Android selects a different hand, mirrors the camera differently, or normalizes landmarks differently, the same sign may become a different model input. Therefore, Python and Android must share the same input shapes, label order, landmark normalization, dominant-hand policy, and post-processing thresholds.

Avatar and Speech Communication Module

The text-to-speech module speaks confirmed recognized text so the hearing or non-signing user can understand the sign language user's input. The speech-to-text module allows the hearing user to respond through spoken language, which is converted into readable text.

The avatar module provides a visual response for the signing user. Known words can trigger lightweight animation files, while unknown words can be fingerspelled. The avatar is downstream of the token composer. This means raw predictions, rejected predictions, and unstable predictions must not trigger avatar animation. Only confirmed accepted tokens should be passed to the avatar.

The current avatar module is a prototype. It does not claim full ASL grammar generation. Its role is to provide basic visual feedback for the controlled demo vocabulary and to show the feasibility of a bidirectional communication interface.

Testing Procedure

Testing is divided into static testing, dynamic testing, negative/no-output testing, Android readiness testing, and user evaluation preparation.

Static Recognition Testing

Static recognition testing checks whether A-Z, del, space, and nothing can be recognized reliably. The researchers use live alphabet testing and diagnostic tools to identify weak static signs. When weak signs are found, additional static calibration samples may be recorded and used to retrain the static landmark model.

Dynamic Word Testing

Dynamic testing checks whether the current 10 word signs can be recognized in live camera conditions. The system records confidence, margin, motion, wrist path, hand presence, accepted or rejected status, and confusion cases. This helps the researchers

determine whether a word needs additional samples, threshold tuning, or cleaner meta-data.

Negative Class Testing

Negative testing verifies whether the system rejects idle hands, open hands, partial signs, transition movement, and aborted signs. These samples are treated as NOTHING. Successful negative-class behavior means that the system does not produce a random word when the user is not signing a valid word.

Android Readiness Testing

Android readiness testing verifies whether the exported model files, label files, runtime manifest, and feature rules can be integrated into the Android prototype. The Android developer must follow the same feature extraction and inference contract used in the Python prototype.

Evaluation Method

The completed VoxGest prototype will be evaluated using selected ISO/IEC 25010 quality characteristics. The criteria are Functional Suitability, Performance Efficiency, Reliability, and Usability.

1. **Functional Suitability** evaluates whether VoxGest performs its required communication functions, including recognition, text output, speech output, speech-to-text, and avatar response.
2. **Performance Efficiency** evaluates whether the system responds fast enough for real-time communication and runs suitably on mobile hardware.
3. **Reliability** evaluates whether the system produces stable outputs, handles missing landmarks, and reduces false recognition.
4. **Usability** evaluates whether the interface is understandable and usable by target users and evaluators.

Respondents of the Study

The study will involve two main respondent groups. IT professionals will evaluate the technical quality of the system, including functionality, performance, reliability, and implementation feasibility. Deaf, mute, hard-of-hearing, signing users, or representative users will evaluate usability, communication effectiveness, readability, accessibility, and user experience.

The final number of respondents may depend on availability and adviser approval. The recommended minimum target is to include IT professionals for technical evaluation and target or representative users for accessibility and usability evaluation.

Data Gathering Instrument

The main data gathering instrument will be a structured questionnaire based on ISO/IEC 25010. The questionnaire will use a 5-point Likert scale.

Table 3.1: Likert Scale Interpretation

Weight	Range	Interpretation
5	4.50 - 5.00	Excellent / Strongly Agree
4	3.50 - 4.49	Very Good / Agree
3	2.50 - 3.49	Good / Neutral
2	1.50 - 2.49	Fair / Disagree
1	1.00 - 1.49	Poor / Strongly Disagree

The questionnaire items will be grouped according to Functional Suitability, Performance Efficiency, Reliability, and Usability. Additional comments may also be collected to gather qualitative suggestions for improvement.

Statistical Treatment of Data

The evaluation results will be analyzed using descriptive statistics. The weighted mean will be used to summarize the responses for each ISO/IEC 25010 category. The formula for the weighted mean is:

$$\text{Weighted Mean} = \text{Sum of weighted responses} / \text{Total number of responses}$$

The computed mean for each category will be interpreted using the Likert scale table. This will allow the researchers to determine the overall acceptability and software quality of the VoxGest prototype.

Research Ethics

The researchers will ensure that respondents are informed about the purpose of the study, the testing activity, and the use of their feedback. Participation will be voluntary. Respondents may decline or stop participation at any time. Any collected responses will be used only for academic evaluation. If images, videos, or demonstration recordings are collected, the researchers will request consent and avoid publishing personally identifiable data without permission.

Because VoxGest involves camera-based input and accessibility-related users, privacy and respect are important. The system is designed for local recognition, and the recognition prototype does not require cloud processing for the current scope.

Summary of Methodology

This chapter explained the methodology used in the development of VoxGest. The study uses developmental research and Agile Rapid Application Development because the project involves building, testing, and improving a working assistive communication system. The current system architecture uses camera input, MediaPipe landmark extraction, static and dynamic recognition models, post-processing gates, token composition, text output, text-to-speech, speech-to-text, and avatar or fingerspelling response.

The methodology also explained why the project evolved from CNN-based image recognition to landmark-based and temporal recognition. Static signs use 63 hand-landmark features, while dynamic words use 30-frame sequences with 162 pose and dominant-hand features per frame. The NOTHING class, dominant-hand policy, and post-processing

gates are included to support reliability and reduce false outputs. The completed prototype will be evaluated using ISO/IEC 25010 quality characteristics through questionnaires answered by IT professionals and target or representative users.

BIBLIOGRAPHY

Books

Warden, P., & Situnayake, D. (2020). *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media.

Journals, Conference Papers, and Technical Publications

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 265-283.

Alzubaidi, L., Al-Amidie, M., Al-Asadi, A., Humaidi, A. J., Al-Shamma, O., Fadhel, M. A., Zhang, J., Duan, Y., & Farhan, L. (2023). Deep learning-based American Sign Language recognition system using transfer learning. *Sensors*, 23(18), 7970. <https://doi.org/10.3390/s23187970>

Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*.

Bragg, D., Koller, O., Bellard, M., Berke, L., Boudreault, P., Braffort, A., Caselli, N., Huen-erfauth, M., Kacorri, H., Verhoef, T., Vogler, C., & Ringel Morris, M. (2019). Sign language recognition, generation, and translation: An interdisciplinary perspective. *The 21st International ACM SIGACCESS Conference on Computers and Accessibility*, 16-31.

Fallah, M. K., Najafi, M., Gorgin, S., & Lee, J.-A. (2024). An ultra-low-computation model for understanding sign languages. *Expert Systems with Applications*, 249, 123782. <https://doi.org/10.1016/j.eswa.2024.123782>

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.

ISO/IEC. (2011). *ISO/IEC 25010:2011 Systems and software engineering - Systems and*

software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. International Organization for Standardization.

Kasapbasi, A., Elbushra, A. E. A., Al-Hardanee, O., & Yilmaz, A. (2022). DeepASLR: A CNN based human computer interface for American Sign Language recognition for hearing-impaired individuals. *Computer Methods and Programs in Biomedicine Update*, 2, 100048. <https://doi.org/10.1016/j.cmpbup.2021.100048>

Li, D., Rodriguez Opazo, C., Yu, X., & Li, H. (2020). Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 1459-1469.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*.

Luqman, H. (2022). An efficient two-stream network for isolated sign language recognition using accumulative video motion. *IEEE Access*, 10, 93785-93798. <https://doi.org/10.1109/ACCESS.2022.3204001>

Rastgoo, R., Kiani, K., & Escalera, S. (2021). Sign language recognition: A deep survey. *Expert Systems with Applications*, 164, 113794. <https://doi.org/10.1016/j.eswa.2020.113794>

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*.

Zhang, Y., Han, Y., Zhu, Z., Jiang, X., & Zhang, Y. (2024). Artificial intelligence in sign language recognition: A comprehensive bibliometric and visual analysis. *Computers and Electrical Engineering*, 120, 109854. <https://doi.org/10.1016/j.compeleceng.2024.109854>

Zhang, Y., & Jiang, X. (2024). Recent advances on deep learning for sign language

recognition. *Computer Modeling in Engineering & Sciences*, 139. <https://doi.org/10.32604/cmes.2023.045731>

Electronic Sources

Google AI Edge. (2025). *MediaPipe solutions guide*. Retrieved from <https://ai.google.dev/edge/mediapipe/solutions/guide>

Google AI Edge. (2025). *LiteRT and on-device machine learning documentation*. Retrieved from <https://ai.google.dev/edge/litert>